



برنامه نویسی شی گرا

Object Oriented Programming

مدرس:

مریم خراشادیزاده

تحلیل و طراحی شی گرا (OOAD)

- برای ایجاد سامانه‌های بزرگ و پیچیده کامپیوتری، برنامه‌نویسان نیازمند فرایندهای مهمتری قبل از رسیدن به کدنویسی هستند.
- یکی از مهمترین مراحل قبل از نوشتن کد یک برنامه، فرایند **تحلیل (Analysis)** است. یعنی باید با دنبال کردن فرایند تجزیه و تحلیلی دقیق، نیازمندیها و الزامات سامانه تعیین شود (به عبارت دیگر سامانه در نظر دارد **چه چیزی** را انجام دهد).
- پس از شناخت نیازمندیهای سامانه، باید راهکاری برای برآورده شدن این نیازمندیها **طراحی (Design)** کرد. (سامانه در نظر دارد **چگونه** این نیازمندیها را عملی کند).
- در حالت ایده‌آل باید این مراحل به دقت طی شود و قبل از نوشتن هر کدی، طراحی انجام شده به دقت بررسی شود؛ حتی در پروژه‌های بزرگ و حیاتی ممکن است طراحی انجام شده توسط سایر متخصصان نرم‌افزار بازبینی و ارزیابی شود.
- در پروژه‌های بزرگ تیم **تحلیل‌گران (Analyzers)**، **طراحان (Designers)** و **برنامه‌نویسان (Programmers)** می‌توانند متشکل از صدها و هزاران مهندس نرم‌افزار باشند که هر کدام در حیطه خاص خود متخصص و مجرب هستند.
- اگر نگرش شی‌گرا در ایجاد نرم‌افزار مدنظر باشد، مراحل تحلیل و طراحی هم با رویکرد شی‌گرا انجام می‌پذیرد و تحلیل و طراحی شی‌گرا (**Object Oriented Analysis and Design**) نامیده می‌شود.
- زبانهای برنامه‌نویسی شی‌گرایی مثل C++، C#، java و ... به ما امکان پیاده‌سازی سامانه‌هایی که به صورت شی‌گرا طراحی شده‌اند، را می‌دهد.
- استراتژیهای مختلفی برای انجام فرایند تحلیل و طراحی شی‌گرا وجود دارد.





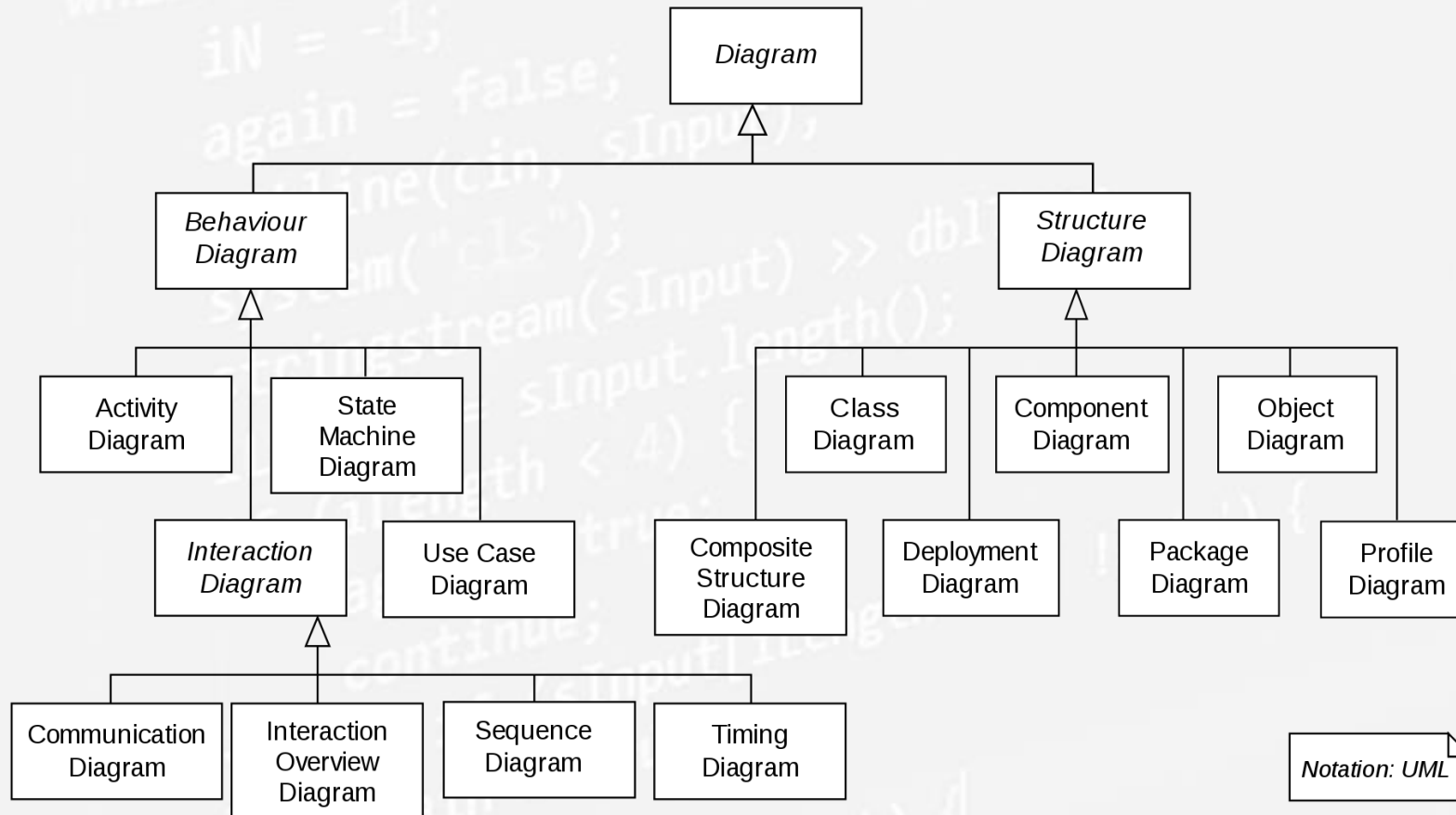
زبان مدلسازی یکپارچه

- علیرغم وجود روشهای مختلف برای انجام فرایند تحلیل و طراحی شیءگرا، یکی از رایجترین و پرکاربردترین روشها برای بیان نتایج تحلیل و طراحی شیءگرا، استفاده از **زبان مدلسازی یکپارچه (Unified Modeling Language)** است.
- **UML** نخستین بار توسط شرکت **rational** در سال ۱۹۹۵ ارائه شد و سپس با استفاده از تجربیات چندتن از دانشمندان در حوزه شیءگرایی تکمیل شد و در نهایت در سال ۱۹۹۷ با ادغام آن با طرحها و ایدههای دیگر، توسط گروه مدیریت شیء (OMG) به عنوان یک زبان مدلسازی یکپارچه ارائه شد که البته از آن زمان تا کنون با ارائه نسخه‌های مختلف دستخوش تغییراتی شده است.
- **UML** یک زبان گرافیکی است که با استفاده از نمودارهای مختلف ساختار و رفتار یک سامانه را مدل می‌کند. هر نمودار در **UML** سامانه را از یک دیدگاه مدل می‌کند.
- نمودارهای **UML** به تحلیلگران و طراحان کمک می‌کند تا یک سامانه کامپیوتری را به صورت خلاصه، گرافیکی و مستقل از زبان برنامه‌نویسی ارائه دهند (قبل از اینکه برنامه‌نویسان آن را با یک زبان برنامه‌نویسی پیاده‌سازی کنند).



زبان مدلسازی یکپارچه

• شکل زیر نمودارهای مختلف زبان **UML** را نشان می‌دهد.



زبان مدل سازی یکپارچه

- یکی از نمودارهای UML که یک دید ساختاری و ایستا از سامانه نرم افزاری نشان می دهد، **نمودار کلاس** است که کلاسهای مربوط به سامانه و ارتباط بین آنها را نشان می دهد.
- این نمودار یکی از نمودارهای مهم UML است که یکی از مهمترین مفاهیم برنامه نویسی شی گرا یعنی کلاسها را در برنامه نشان می دهد و به راحتی می تواند به زبان برنامه نویسی شی گرا مثل C++ نگاشت شود. این نمودار ویژگیها و عملیات مربوط به هر کلاس را به صورت مختصر نشان می دهد.
- در این نمودار یک کلاس توسط یک **مستطیل** نشان داده می شود که دارای سه بخش است:
 - بخش بالایی شامل **نام کلاس** است.
 - بخش میانی شامل **صفات کلاس** است که همان اعضای داده ای در کلاسهای C++ است.
 - بخش پایینی شامل **لیست متدهای مرتبط با کلاس** است که در واقع همان توابع عضو در کلاسهای C++ را نشان می دهد.
- در این نمودار می توان سطح دسترسی، نوع داده و مقدار اولیه اعضای داده ای و همچنین **prototype** توابع را مشخص کرد.
 - سطح دسترسی **عمومی** با نماد + در کنار عضو کلاس نشان داده می شود.
 - سطح دسترسی **حفاظت شده** با نماد # در کنار عضو کلاس نشان داده می شود.
 - سطح دسترسی **خصوصی** با نماد - در کنار عضو کلاس نشان داده می شود.



زبان مدلسازی یکپارچه

• شکل زیر کلاس دایره را نشان می‌دهد.

Circle
Raduis
getArea
getPerimeter

Circle
- Radius:integer
+«constructor» Circle(radius:integer)
+«destructor»~Circle()
+setRadius(radius:integer)
+getRadius():integer
+getArea():float
+getPerimeter():float



مثال ۱:



- می‌خواهیم کلاس Point را برای نگهداری مختصات نقاط صفحه ایجاد کنیم و توابع مقداردهی، دستیابی، سازنده، مخرب و نمایش مختصات به فرم (x, y) را برای آن پیاده‌سازی کنیم.

Point
-x:integer -y:integer
+«constructor» Point(a:integer,b:integer) +«destructor» ~Point() +setX(a:integer) +getX():integer +setY(a:integer) +getY():integer +display()



مثال ۱: کلاس Point

```
#include <iostream>

using namespace std;

class Point
{
public:
    Point(int = 0,int = 0);
    ~Point();

    int Getx() { return x; }
    void Setx(int val) { x = val; }
    int Gety() { return y; }
    void Sety(int val) { y = val; }
    void display();

protected:

private:
    int x;
    int y;
};

Point::Point(int a,int b):x(a),y(b)
{
    cout << "(" << x << ", " << y << ")" << " is constructed!" << endl;
}

Point::~~Point()
{
    cout << "(" << x << ", " << y << ")" << " is destructed!" << endl;
}

void Point::display()
{
    cout << "(" << x << ", " << y << ")" << endl;
}

int main()
{
    Point p1,p2(10,20);
    p1.Setx(100);
    p1.Sety(200);
    p1.display();
    p2.display();

    return 0;
}
```

```
"C:\Users\Maryam\courses\Advanced...
(0,0) is constructed!
(10,20) is constructed!
(100,200)
(10,20)
(10,20) is destructed!
(100,200) is destructed!

Process returned 0 (0x0)   execution time : 0.453 s
Press any key to continue.
```



اعضای داده‌ای ثابت در کلاس

- می‌توان در کلاس اعضای داده‌ای ثابت تعریف کرد.
- برای این کار کافی است کلمه کلیدی «**const**» را قبل از نوع داده آورد.
- برای مقداردهی اولیه اعضای داده‌ای ثابت هم از سازنده استفاده می‌شود، با این تفاوت که مقداردهی نمی‌تواند در بدنه تابع سازنده صورت گیرد.
- برای مقداردهی اولیه اعضای داده‌ای ثابت کلاس باید بعد از لیست پارامترهای سازنده و قبل از بدنه تابع، عملگر **:** و به دنبال آن نام متغیر و درون پرانتز، مقدار آن آورده شود.
- برای مقداردهی اولیه متغیرهای ارجاع (متغیرهایی که با **&** تعریف می‌شوند) نیز باید به این شیوه عمل کرد.
- این شیوه مقداردهی را برای بقیه اعضای داده‌ای غیرثابت هم می‌توان به کار برد. برای جدا کردن مقداردهی چندین عضو داده‌ای به این شیوه از **,** استفاده می‌شود.



مثال ۲: اعضای داده‌ای ثابت در کلاس

```
3 using namespace std;
4 class VerticalLinesPoint
5 {
6     public:
7         VerticalLinesPoint(int = 0,int = 0);
8         ~VerticalLinesPoint();
9
10        int Getx(){ return x; }
11        int Gety() { return y; }
12        void Sety(int val) { y = val; }
13        void display();
14
15        protected:
16
17        private:
18            const int x;
19            int y;
20    };
21
22    VerticalLinesPoint::VerticalLinesPoint(int a,int b):x(a)
23    {
24        // x = a; Error
25        y = b;
26        cout << "("<<x<<','<<y<<") is constructed!"<<endl;
27    }
28
29    VerticalLinesPoint::~~VerticalLinesPoint()
30    {
31        cout << "("<<x<<','<<y<<") is destructed!"<<endl;
32    }
33
34    void VerticalLinesPoint::display()
35    {
36        cout << '(' << x << ',' << y << ')' << endl;
37    }
38
39
40    int main()
41    {
42        cout<<"Begin of main" <<endl;
43
44        VerticalLinesPoint p1,p2(10,20),p3(40);
45        p1.Sety(200); p3.Sety(50);
46        p1.display(); p2.display(); p3.display();
47
48        cout << "End of main" << endl;
49        return 0;
50    }
```

← معرفی عضو داده‌ای ثابت

← مقداردهی اولیه به عضو داده‌ای ثابت

```
"C:\Users\Maryam\courses\Advanced programing\Line\bin...
Begin of main
(0,0) is constructed!
(10,20) is constructed!
(40,0) is constructed!
(0,200)
(10,20)
(40,50)
End of main
(40,50) is destructed!
(10,20) is destructed!
(0,200) is destructed!
Process returned 0 (0x0)   execution time : 0.406 s
Press any key to continue.
```



اشیای ثابت و توابع ثابت عضو کلاس

- بعضی از اشیای ایجاد شده از کلاس نیاز دارند تغییر کنند و بعضی از اشیای بعد از ایجاد نیازی به تغییر در محتوای داده‌ای خود ندارند. بنابراین برای جلوگیری از خطاها می‌توان **اشیای ثابت** از کلاس ایجاد کرد.
- برای این کار کافی است کلمه کلیدی «**const**» را قبل از نام کلاس آورد.
- در C++ تنها می‌توان توابعی از کلاس را برای اشیای ثابت فراخوانی کرد، که خود به صورت ثابت تعریف شده باشد. به عبارت دیگر حتی اگر تابع مقدار داده‌ها را تغییر ندهد، ولی به صورت ثابت تعریف نشده باشد، نمی‌توان برای اشیای ثابت آن را فراخوانی کرد.
- **تابع ثابت** تابعی است که هم در prototype آن بعد از لیست پارامترها و هم در پیاده‌سازی تابع بعد از لیست پارامترها و قبل از بدنه تابع کلمه کلیدی «**const**» آورده شود.
- **توابع ثابت** نمی‌توانند مقادیر داده‌ای کلاس را تغییر دهند.
- **توابع ثابت** نمی‌توانند توابعی که مقادیر داده‌ای کلاس را تغییر می‌دهند، را فراخوانی کنند.
- **تابع سازنده و نابودکننده** می‌توانند علیرغم اینکه داده‌ها را تغییر می‌دهند، برای اشیای ثابت فراخوانی شوند و نباید آن را ثابت تعریف کرد.



مثال ۳: اشیای ثابت و توابع ثابت عضو کلاس

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Point
6  {
7  public:
8      Point(int = 0, int = 0);
9      ~Point();
10
11      int Getx() const { return x; }
12      void Setx(int val) { x = val; }
13      int Gety() const { return y; }
14      void Sety(int val) { y = val; }
15      void display() const;
16
17  protected:
18
19  private:
20      int x;
21      int y;
22  };
23
24  Point::Point(int a, int b) : x(a), y(b)
25  {
26      cout << "(" << x << ", " << y << ") is constructed!" << endl;
27  }
28
29  Point::~~Point()
30  {
31      cout << "(" << x << ", " << y << ") is destructed!" << endl;
32  }
33
34  void Point::display() const
35  {
36      cout << "(" << x << ", " << y << ") " << endl;
37  }
38
39  int main()
40  {
41      cout << "begin of main" << endl;
42      Point p1, p2(10, 20);
43      const Point Origin;
44      p1.Setx(100);
45      p1.Sety(200);
46      // Origin.Setx(30); Error
47      p1.display();
48      p2.display();
49      Origin.display();
50
51      cout << "end of main" << endl;
52
53      return 0;
54  }
```

توابع ثابت

پایده‌سازی توابع ثابت

اعلان شی ثابت

```
"C:\Users\Maryam\courses\Advanced programing\point\bi...
begin of main
(0,0) is constructed!
(10,20) is constructed!
(0,0) is constructed!
(100,200)
(10,20)
(0,0)
end of main
(0,0) is destructed!
(10,20) is destructed!
(100,200) is destructed!

Process returned 0 (0x0)   execution time : 0.728 s
Press any key to continue.
```



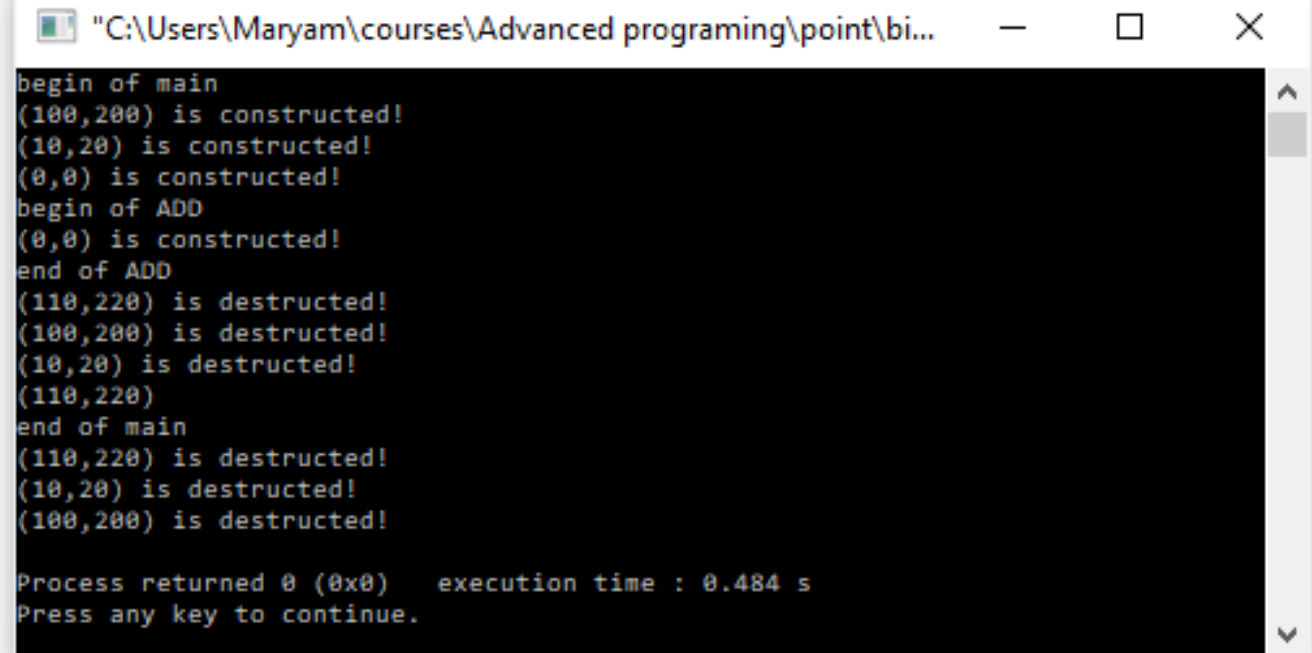
توابع و اشیا

- می توان اشیا را به عنوان پارامتر به توابع ارسال کرد.
- نوع داده خروجی یک تابع نیز می تواند یک شی باشد.
- می توان در توابع متغیرهای محلی از نوع کلاس تعریف کرد.
- بدیهی است که دسترسی به اعضای خصوصی کلاس در تابع مستقل امکانپذیر نیست و یکی از راهکارهای دسترسی به اعضای خصوصی استفاده از توابع دستیابی و مقداردهی است.
- عملگر انتساب به صورت پیش فرض در C++ تعریف شده است و بخش داده ای دو شی را نظیر به نظیر نسبت می دهد.
- در صورت ارسال اشیا به تابع یا تعریف متغیرهای محلی فراخوانیهای سازنده و نابود کننده چگونه خواهد بود؟ اگر فراخوانی با مقدار یا فراخوانی با ارجاع صورت گیرد، آیا این فراخوانیها تغییر خواهد کرد؟



مثال ۴: ارسال اشیاء به تابع با مقدار

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  Point ADD(Point ,Point );
7
8  int main()
9  {
10     cout << "begin of main" << endl;
11     Point point1(100,200),point2(10,20),sum;
12     sum = ADD(point1,point2);
13     sum.display();
14     cout << "end of main" << endl;
15
16     return 0;
17 }
18
19 Point ADD(Point p1,Point p2)
20 {
21     cout << "begin of ADD" << endl;
22     Point p3;
23     p3.Setx(p1.Getx()+p2.Getx());
24     p3.Sety(p1.Gety()+p2.Gety());
25     cout<<"end of ADD" << endl;
26     return p3;
27 }
28
```



```
"C:\Users\Maryam\courses\Advanced programming\point\bi...
begin of main
(100,200) is constructed!
(10,20) is constructed!
(0,0) is constructed!
begin of ADD
(0,0) is constructed!
end of ADD
(110,220) is constructed!
(100,200) is destroyed!
(10,20) is destroyed!
(110,220)
end of main
(110,220) is destroyed!
(10,20) is destroyed!
(100,200) is destroyed!

Process returned 0 (0x0)   execution time : 0.484 s
Press any key to continue.
```


مثال ۵: ارسال اشیا به تابع با ارجاع

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  Point ADD(Point &, Point &);
7
8  int main()
9  {
10     cout << "begin of main" << endl;
11     Point point1(100,200), point2(10,20), sum;
12     sum = ADD(point1, point2);
13     sum.display();
14     cout << "end of main" << endl;
15
16     return 0;
17 }
18
19 Point ADD(Point &p1, Point &p2)
20 {
21     cout << "begin of ADD" << endl;
22     Point p3;
23     p3.Setx(p1.Getx()+p2.Getx());
24     p3.Sety(p1.Gety()+p2.Gety());
25     cout<<"end of ADD" << endl;
26     return p3;
27 }
28
```

مثال ۶: ارسال اشیا به تابع با اشاره گر

```
17 string sInput;  
18 int iLength, iN;  
19 double dblTemp;  
20 bool again = true;  
21  
22 while (again) {  
23     iN = -1;  
24     again = false;  
25     getline(cin, sInput);  
26     system("cls");  
27     stringstream(sInput) >> dblTemp;  
28     iLength = sInput.length();  
29     if (iLength < 4) {  
30         again = true;  
31         continue;  
32     } else if (sInput[iLength - 3] != '.') {  
33         again = true;  
34     }  
35 }
```

